Guion del proyecto final: Sistema web para análisis inteligente de imágenes en AWS

Para: Cristina, Isidro, Jorge, Edwin y Kris **Presentación:** miércoles 11 de noviembre de 2026, 7:00 p. m. Vestimenta formal.

Este guion explica qué se hizo, cómo se hizo y por qué se hizo así, componente por componente. Cada quien tiene su parte, pero **el catedrático puede preguntarle a cualquiera sobre cualquier capa**, así que lean todo, no solo lo suyo.

Enlaces

Qué	Dónde
Sistema en vivo	http://imgproj-alb-399228811.us-east-1.elb.amazonaws.com
Código, informe y presentación	https://github.com/Krisggx/Proyecto-Final-CloudComputing
Informe final (APA 7)	`docs/Informe_Final_Cloud_Computing.pdf` en el repositorio
Guía técnica	`DESPLIEGUE.md` en el repositorio

Si Chrome bloquea el sitio por ser `http`, ábranlo en Edge, en el celular o en una ventana de incógnito.

---

1. El proyecto en 30 segundos

Construimos una página web donde subes una foto. La foto se guarda en **Amazon S3**, **Amazon Rekognition** dice qué contiene (por ejemplo "Sphere 100 %, Purple 99 %") y el resultado queda guardado en una base de datos **Amazon RDS (PostgreSQL)**. Hay un historial de todas las imágenes, que también se puede borrar.

Lo importante del curso es la **alta disponibilidad**: hay **dos copias de la aplicación** en dos zonas distintas de AWS, con **dos niveles de balanceo** (un ALB y dos HAProxy). Si apagamos un servidor, el sitio sigue funcionando. Lo probamos en vivo y lo medimos.

Toda la infraestructura (39 recursos de AWS) se crea con **una sola plantilla de CloudFormation**. Eso es *Infraestructura como Código*, uno de los puntos extra.

2. Cómo viaja una imagen (todos deben saber explicarlo)

1. El usuario entra por el **Application Load Balancer (ALB)**, que es público y está en 2 zonas.
2. El ALB manda la solicitud a uno de los dos **HAProxy** (uno en cada zona).
3. El HAProxy la manda a una de las dos **réplicas de la aplicación** (`app-a` o `app-b`).
4. La aplicación **valida** el archivo: tiene que ser JPG o PNG de verdad (se revisan los primeros bytes, no solo el nombre) y pesar 5 MB o menos.
5. La aplicación **guarda** la imagen en S3 con un nombre único (UUID).
6. La aplicación le pide a **Rekognition** (operación `DetectLabels`) las etiquetas con confianza de 70 % o más.
7. La aplicación **registra** en RDS la imagen, las etiquetas, la confianza y la fecha.
8. La aplicación **muestra** el resultado. La imagen se ve con un enlace temporal de 15 minutos, así el bucket nunca es público.

Los **5 servidores** que pide el curso son: `haproxy-a`, `haproxy-b`, `app-a`, `app-b` y `bastion`. RDS no cuenta como servidor porque es un servicio administrado.

3. Orden de la presentación

#	Diapositiva	Presenta	Tiempo
1	Portada	Cristina	20 s
2	Equipo	Cristina (cada quien dice su nombre y su componente)	40 s
3	Problema y objetivo	Cristina	1 min
4	Arquitectura	Cristina e Isidro	2 min
5	Flujo de una imagen	Jorge	1 min
6	La aplicación	Jorge	1 min
7	Tecnologías	Jorge	1 min
8	Alta disponibilidad	Isidro	1,5 min
9	Seguridad	Cristina	1 min
10	Infraestructura como código	Kris	1 min
11	Demostración en vivo	Isidro, Jorge y Kris	5 min
12	Resultados	Edwin	1,5 min
13	Recuperación y respaldos	Edwin	1,5 min

#	Diapositiva	Presenta	Tiempo
14	Conclusiones	Kris	1 min
15	Preguntas	Todos	—

---

4. Parte de cada integrante

Cristina Marie Jimenez Garcia (20680): red, grupos de seguridad y ALB

Qué se hizo

- Una red privada (VPC `10.0.0.0/16`) con **6 subredes** en 2 zonas: 2 públicas (HAProxy y bastión), 2 privadas para la aplicación y 2 privadas para la base de datos.
- Un **NAT Gateway**, para que las réplicas privadas descarguen paquetes sin ser accesibles desde internet.
- Un **VPC endpoint de S3**, para que el tráfico hacia S3 no salga a internet.
- **5 grupos de seguridad encadenados**: ALB → HAProxy → App → RDS. Cada capa solo acepta tráfico de la capa anterior.
- El **Application Load Balancer** público, con un *target group* que contiene a los 2 HAProxy y revisa `/lb-health` cada 5 segundos.

Cómo se hizo: todo está declarado en la plantilla `infra/imgproj-stack.yaml`, en los recursos `VPC`, `*Subnet*`, `NatGateway`, `*SG`, `LoadBalancer` y `HAProxyTargetGroup`.

Por qué se hizo así

- Usamos **subredes privadas** porque la aplicación y la base de datos no deben poder alcanzarse desde internet. Es uno de los controles de seguridad que pide el curso.
- **Encadenamos los grupos de seguridad** por privilegio mínimo: el puerto 5432 de la base solo lo puede usar la aplicación (y el bastión, para administrarla).
- Usamos **dos zonas** porque, si se cae toda una zona de AWS, la otra sigue atendiendo.
- Pusimos el **ALB delante de HAProxy** porque da una sola dirección pública en 2 zonas y retira solo al HAProxy que falle.

Qué decir: "La red está dividida en capas: lo público es solo el balanceador. La aplicación y la base de datos están en subredes privadas, y cada capa solo acepta tráfico de la anterior."

Preguntas que te pueden hacer

- *¿Por qué ALB y HAProxy, si los dos balancean?* HAProxy es obligatorio en el curso y balancea entre las réplicas con comprobaciones y reintentos. El ALB da la entrada pública en 2 zonas y

protege contra la caída de un HAProxy. Así ninguna capa depende de un solo servidor.

- *¿Qué pasa si se cae una zona?* El ALB deja de mandar tráfico al HAProxy de esa zona y el otro HAProxy atiende con la réplica que queda.
- *¿Cuál es un punto débil de la red?* Hay un solo NAT Gateway. Si falla, las réplicas no pueden descargar paquetes, pero siguen atendiendo. Está en las recomendaciones.

Isidro Alexander Chuj Quijivix (231621): HAProxy y comprobaciones de estado

Qué se hizo

- **Dos nodos HAProxy**, uno por zona, con la misma configuración (`haproxy/haproxy.cfg`).
- Cada nodo reparte las solicitudes entre `app-a` y `app-b` por turnos (*round robin*).
- Cada nodo revisa `/health` en cada réplica cada 5 segundos y la marca DOWN después de 3 fallos.
- `retries 2` y `option redispatch`: si una conexión falla, HAProxy la reintenta en la otra réplica.
- `monitor-uri /lb-health`, que es lo que revisa el ALB para saber si el HAProxy está vivo.
- Una cabecera `X-LB-Node` que dice qué HAProxy respondió, y una página de estadísticas en el puerto 8404, accesible solo desde el bastión.

Cómo se hizo: CloudFormation instala HAProxy en cada servidor al arrancar, escribe la configuración con las IP privadas de las réplicas y la valida con `haproxy -c` antes de iniciar el servicio.

Por qué se hizo así

- Usamos **dos nodos** porque un solo HAProxy sería un punto único de falla.
- Usamos **reintentos con redispatch** porque, gracias a ellos, apagar una réplica no produjo **ni un solo error** (prueba P-08).
- La **cabecera** `X-LB-Node` sirve como evidencia del balanceo durante la demo.

Qué decir: "Hay dos niveles de balanceo. El ALB vigila a los HAProxy y los HAProxy vigilan a las réplicas. Cuando una réplica cae, HAProxy reintenta en la otra y el usuario no ve el error."

En la demo: explicas la prueba de apagar `app-a` mientras Kris ejecuta el comando.

Preguntas que te pueden hacer

- *¿Por qué al apagar un HAProxy sí hubo 5 errores 502?* El ALB tarda unos 9 segundos (2 comprobaciones de 5 s) en notar que el HAProxy cayó. Mientras tanto le sigue mandando algunas solicitudes. La forma correcta de hacer mantenimiento es quitarlo primero del target group (*Deregister*). Así lo probamos y hubo 0 errores de 66.
- *¿Qué es round robin?* Repartir las solicitudes por turnos: una a `app-a`, otra a `app-b`, y así.
- *¿Cada cuánto revisa HAProxy?* Cada 5 segundos; marca DOWN con 3 fallos seguidos y vuelve a UP con 2 éxitos.

Jorge José Adrián Monterroso Flores (18610): aplicación, S3 y Rekognition

Qué se hizo

- La aplicación web en **Python con Flask**, servida con **Gunicorn** (3 trabajadores) como servicio de Linux (systemd).
- **Validación de archivos en el servidor**: extensión, firma real del archivo (JPG o PNG) y máximo 5 MB.
- **Integración con S3**, para guardar con nombre UUID y mostrar con enlaces temporales de 15 minutos.
- **Integración con Rekognition** (`DetectLabels`, máximo 10 etiquetas, confianza de 70 % o más).
- Las páginas de **carga** (con zona para arrastrar y vista previa), **resultado** (con barras de confianza) e **historial** (en galería).
- El botón **"Borrar historial"**, que pide confirmación y borra los registros de la base. Las imágenes quedan en S3 como respaldo.
- **10 pruebas automáticas** (`tests/test_app.py`).

Cómo se hizo: el código está en `app/app.py`. La aplicación no tiene ninguna contraseña escrita: usa el **rol de IAM** de la instancia para S3 y Rekognition, y lee la contraseña de la base desde **Secrets Manager**.

Por qué se hizo así

- Usamos **Flask** porque es simple y suficiente para 4 páginas, y **Boto3** porque es el SDK oficial de AWS para Python.
- **Validamos en el servidor** porque la validación del navegador se puede saltar. Revisar los primeros bytes evita que un PDF renombrado a `.jpg` pase (prueba P-05).
- Usamos **nombres UUID** para que nadie pueda sobrescribir imágenes ni manipular rutas.
- Usamos **enlaces temporales** para que el bucket nunca sea público.
- El **borrado no elimina las imágenes de S3** porque el rol no tiene permiso de borrado. Es privilegio mínimo, y además sirve de respaldo.

Qué decir: "La aplicación es la que une todo: valida, guarda en S3, consulta a Rekognition, registra en RDS y muestra el resultado. No tiene credenciales en el código."

En la demo: subes una imagen y muestras las etiquetas y el historial. Después intentas subir el PDF renombrado y la imagen de más de 5 MB. Al final borras el historial y vuelves a analizar una imagen.

Preguntas que te pueden hacer

- *¿Cómo sabe la aplicación que una imagen es JPG de verdad?* Revisa la firma de los primeros bytes: JPG empieza con `FF D8 FF` y PNG con `89 50 4E 47`.

- *¿Por qué 70 % de confianza?* Para mostrar solo etiquetas confiables. El valor se puede cambiar con una variable de entorno.
- *¿Cómo se conecta a la base sin contraseña en el código?* Al iniciar, le pide el secreto a Secrets Manager usando el rol de la instancia.
- *¿Qué hace exactamente "Borrar historial"?* Ejecuta `DELETE FROM images`; las etiquetas se borran en cascada. Lo probamos en el sitio real: se borraron 14 registros desde `app-a` y 2 desde `app-b`.

Edwin Manuel Valenzuela Chaj (231715): RDS, respaldos y recuperación

Qué se hizo

- Una base **Amazon RDS for PostgreSQL** (versión 18.3, `db.t3.micro`) en subredes privadas, **sin acceso público** y con almacenamiento **cifrado**.
- Una **contraseña generada automáticamente** por Secrets Manager. Nadie del equipo la conoce.
- Un esquema de dos tablas: `images` (id, nombre, clave en S3, fecha) y `labels` (nombre, confianza), relacionadas con borrado en cascada.
- **Respaldos automáticos** de 7 días, con restauración a cualquier punto en el tiempo, y un **snapshot manual** antes de la demo.
- **Respaldo lógico diario** con `pg_dump` desde el bastión hacia S3 (`backups/`).
- La **guía de recuperación** de cada componente: sección "Guía de recuperación" del informe y `DESPLIEGUE.md`.

Por qué se hizo así

- Elegimos **RDS en lugar de DynamoDB** porque el historial se consulta ordenado por fecha, y eso es natural en SQL. Además, los snapshots facilitan demostrar respaldo y restauración, y la subred privada demuestra el acceso privado a la base.
- Usamos **Secrets Manager** para que la contraseña nunca aparezca en el código, en el informe ni en GitHub.
- Hay **dos tipos de respaldo** porque los de RDS recuperan toda la base y el `pg_dump` permite revisar o mover los datos.

Qué decir: "La base de datos es privada: desde internet no se puede ni conectar (prueba P-11). Tenemos respaldos automáticos, un snapshot manual y un respaldo lógico diario, y cada componente tiene su procedimiento de recuperación."

Preguntas que te pueden hacer

- *¿Qué pasa si se cae la base?* Las dos réplicas responden 503 en `/health` y no se pueden subir imágenes. Se recupera reiniciando o restaurando a un punto en el tiempo. Es el componente

con menos redundancia; por eso la primera recomendación es activar **Multi-AZ**.

- *¿Cómo se restaura?* En la consola de RDS: *Restore to point in time*. Luego se cambia `DB_HOST` en las réplicas y se recarga el servicio.
- *¿Dónde está la contraseña?* En Secrets Manager. La aplicación y el bastión la leen con su rol; ninguna persona la escribió.
- *¿Cómo demuestran que la base no es pública?* El endpoint resuelve a una IP privada (10.0.21.108) y una conexión desde internet al puerto 5432 no responde.

Kris Gregorio Garcia Xep (23259): bastión, CloudFormation, pruebas y documentación

Qué se hizo

- La **plantilla de CloudFormation** que crea los **39 recursos**: red, 5 servidores, ALB, S3, RDS, secreto y rol de IAM.
- Los servidores **se configuran solos** al arrancar (*user data*): instalan HAProxy o la aplicación, escriben la configuración y levantan los servicios.
- El **bastión**, con comandos listos: `imgproj-psql` para consultar la base, `imgproj-backup` para el respaldo y `imgproj-continuidad` para la prueba de continuidad.
- **Administración sin llaves SSH**: todo se hace con **Systems Manager** desde CloudShell.
- Las **pruebas de continuidad**: un registro de respuestas del ALB cada medio segundo mientras se apagan servidores.
- Las **actualizaciones escalonadas** de la aplicación, una réplica a la vez. Con reinicio hubo 1 error de 196 solicitudes; con **recarga suave** (`systemctl reload`, señal HUP) hubo **0 errores de 133**.
- El informe en formato APA 7, la presentación y el repositorio de GitHub.

Por qué se hizo así

- Usamos **Infraestructura como Código** porque todo el sistema se puede recrear en unos 20 minutos con un archivo, sin pasos manuales ni olvidos. Es un punto extra del curso.
- Elegimos **CloudFormation en lugar de Terraform** porque viene integrado en AWS, no requiere instalar nada y la plantilla se sube desde la consola.
- Usamos **Systems Manager en lugar de SSH** porque no hay que guardar llaves privadas ni abrir puertos extra, y es más seguro.
- Usamos la **recarga suave** porque Gunicorn arranca trabajadores con el código nuevo y deja terminar las solicitudes en curso, así que el usuario no nota la actualización.

Qué decir: "Toda la infraestructura está en una plantilla. Si se pierde un servidor, *Update stack* lo recrea. Además medimos todo: cada prueba tiene su registro y su captura."

En la demo: ejecutas los comandos en CloudShell (sección 5).

Preguntas que te pueden hacer

- *¿Qué es user data?* Un script que EC2 ejecuta una sola vez, la primera vez que arranca la instancia. Ahí se instala y configura todo.
- *¿Cómo actualizaron la aplicación sin apagar el sitio?* Una réplica a la vez. Mientras una se recarga, la otra atiende, y la recarga suave no corta las conexiones.
- *¿Para qué sirve el bastión si usan Systems Manager?* Para administrar la base (consultas y respaldos con `pg_dump`) desde dentro de la red, porque RDS no es accesible desde fuera.

---

5. Guion de la demostración (5 minutos)

Antes de empezar, dejar abiertas estas pestañas:

- El sitio.
- La consola de S3, en el bucket `imgproj-imagesbucket-…`, carpeta `uploads/`.
- EC2 → Grupos de destino → `imgproj-tg-haproxy` → Destinos.
- CloudShell.

1. **Jorge:** sube una imagen, muestra las etiquetas y su confianza, y abre el historial.
2. **Jorge:** muestra que la imagen está en S3.
3. **Jorge:** intenta subir el PDF renombrado a `.jpg` (error de tipo) y la imagen grande (error de tamaño).
4. **Kris** ejecuta en CloudShell la prueba de continuidad, que pide la página cada segundo y muestra qué HAProxy y qué réplica respondió:

```
A=http://imgproj-alb-399228811.us-east-1.elb.amazonaws.com
for i in $(seq 1 40); do c=$(curl -s -o /dev/null -D h -w '%{http_code}' $A/health); echo
"$(date +%T) HTTP $c $(grep -i x-lb-node h|cut -d' ' -f2|tr -d '\r') $(grep -i x-served-by
h|cut -d' ' -f2|tr -d '\r')"; sleep 1; done
```

5. **Isidro** explica mientras **Kris** apaga `app-a` en otra pestaña de CloudShell. Todo sigue en `HTTP 200`, ahora solo con `app-b`:

```
aws ssm send-command --targets Key=tag:Name,Values=app-a --document-name AWS-RunShellScript --
parameters 'commands=["systemctl stop imgapp"]'
```

6. **Kris** la vuelve a encender:

```
aws ssm send-command --targets Key=tag:Name,Values=app-a --document-name AWS-RunShellScript --
parameters 'commands=["systemctl start imgapp"]'
```

7. **Isidro** muestra el retiro ordenado de un balanceador. En el target group: seleccionar `haproxy-a` → **Anular el registro** (*Deregister*). El sitio sigue respondiendo por `haproxy-b`. Después: **Registrar destinos** → `haproxy-a`, puerto 80.
8. **Jorge** presiona **"Borrar historial"**, confirma y vuelve a analizar una imagen para dejar el historial con datos.

Si algo falla en vivo, muestren las capturas del informe (Figuras 8 a 14): son del mismo sistema y de las mismas pruebas.

6. Preguntas que cualquiera debe poder responder

- **¿Qué hace Rekognition?** Detecta objetos, escenas y conceptos en una imagen y da un porcentaje de confianza, sin que tengamos que entrenar un modelo.
- **¿Dónde se guarda cada cosa?** La imagen en S3; las etiquetas, la confianza y la fecha en RDS; la contraseña en Secrets Manager.
- **¿Qué pasa si se apaga una réplica?** HAProxy la marca DOWN y reintenta en la otra: 0 errores.
- **¿Qué pasa si se apaga un HAProxy?** El ALB lo retira en unos 9 s. Con el retiro ordenado, 0 errores.
- **¿Cuáles son los 4 controles de seguridad?**
- Privilegio mínimo en el rol de IAM.
- Secretos fuera del código, en Secrets Manager.
- Base de datos privada.
- Validación de tipo y tamaño de archivo.
- **¿Qué mejorarían?**
- RDS Multi-AZ.
- Un segundo NAT Gateway.
- Auto Scaling.
- HTTPS con Certificate Manager.
- Usuarios con Cognito.
- **¿Cuánto cuesta?** Los recursos cobran por hora, sobre todo el NAT Gateway, el ALB y RDS. Por eso el stack se borra después de presentar.

7. Lista final antes de las 7:00 p. m.

☐ Cada quien leyó su parte y la sección 6.
☐ El sitio abre (probar en el navegador que se usará para proyectar).
☐ Pestañas de la demo abiertas y sesión de AWS iniciada.

☐ Hay datos en el historial para mostrar.

☐ Presentación en PDF descargada como respaldo.

☐ Vestimenta formal.

☐ Después de la presentación: vaciar el bucket y borrar el stack en CloudFormation.